

1) Basic Student class (Req. 1 & 2)

```
public class Student {
    private static int idCounter = 1001;
    private String uniqueID;
    private String name;
    private int rollNumber;
    private String Faculty;

    public Student (String name, int rollNumber, String Faculty) {
        this.uniqueID = "STU" + idCounter++;
        this.name = name;
        this.rollNumber = rollNumber;
        this.Faculty = Faculty;
    }

    // Getters and Setters
    public String getUniqueID() { return uniqueID; }
    public String getName() { return name; }
    public void setName (String name) { this.name = name; }
    public int getRollNumber() { return rollNumber; }
    public void setRollNumber (int rollNumber) { this.rollNumber = rollNumber; }
    public String getFaculty() { return Faculty; }
    public void setFaculty (String faculty) { this.Faculty = faculty; }

    public void displayProfile () {
        System.out.println ("ID: " + uniqueID + ", Name: " + name +
            ", Roll: " + rollNumber + ", Faculty: " + Faculty);
    }
}
```

2. Employee Hierarchy (Req 2)

```
public class Employee {  
    protected String name;  
    protected String empId;  
    public Employee (String name, String empId) {  
        this.name = name;  
        this.empId = empId;  
    }  
    public void displayDetails () {  
        System.out.println ("Employee ID: " + empId + ", Name: " +  
            name);  
    }  
    class Teacher extends Employee {  
        private String subject;  
        public Teacher (String name, String empId, String subject) {  
            super (name, empId);  
            this.subject = subject;  
        }  
        @Override  
        public void displayDetails () {  
            System.out.println ("Teacher ID: " + empId + " Name: " +  
                name + " subject: " + subject);  
        }  
    }  
    class AdminStaff extends Employee {  
        private String department;  
        public AdminStaff (String name, String empId, String department) {  
            super (name, empId);  
            this.department = department;  
        }  
    }  
}
```

@Override

```
public void displayDetails () {  
    System.out.println ("Admin Staff ID: " + empId + ", Name: " +  
        name + ", Dept: " + department);  
}
```


3. Course Registration (Reg 3)

```
import java.util.ArrayList;
import java.util.List;

public class Course {
    private String courseCode;
    private String courseName;

    public Course(String courseCode, String courseName) {
        this.courseCode = courseCode;
        this.courseName = courseName;
    }

    public String getCourseName() { return courseName; }

    class EnrolledStudents extends Student {
        private List<Course> registeredCourses;

        public EnrolledStudent(String name, int rollNumber, String Faculty) {
            super(name, rollNumber, Faculty);
            this.registeredCourses = new ArrayList<>();
        }

        public void registerCourse(Course course) {
            registeredCourses.add(course);
        }

        public void displayRegisteredCourses() {
            System.out.println("Course registered for " + getName() + " : ");
            for (Course c : registeredCourses) {
                System.out.println("- " + c.getCourseName());
            }
        }
    }
}
```

4. Fee Structure (Req. 4)

```
abstract class StudentFee {
```

```
    protected String name;
```

```
    public abstract double calculateFee();
```

```
}
```

```
class Undergraduate extends StudentFee {
```

```
    @Override
```

```
    public double calculateFee() {
```

```
        return 50000.00;
```

```
}
```

```
}
```

```
class Graduate extends StudentFee {
```

```
    @Override
```

```
    public double calculateFee() {
```

```
        return 80000.00;
```

```
}
```

```
}
```

5. Grading System (Req 5)

interface ResultProcessor {

String processGrade(double marks);

}

class EngineeringDepartment implements ResultProcessor {

@Override

public String processGrade(double marks) {

if (marks >= 80) return "Distinction";

else if (marks >= 60) return "First Class";

return "Pass";

}

}

class ManagementDepartment implements ResultProcessor {

@Override

public String processGrade(double marks) {

if (marks >= 75) return "A Grade";

else if (marks >= 55) return "B Grade";

return "C Grade";

}

}

6. Library Members (Req 6)

```
public class member {
```

```
    protected String memberId;
```

```
    protected String name;
```

```
    public Member (String memberId, String name) {
```

```
        this.memberId = memberId;
```

```
        this.name = name;
```

```
    }
```

```
    public int getBorrowingLimit () {
```

```
        return 2;
```

```
    }
```

```
}
```

```
class StudentMember extends member {
```

```
    public StudentMember (String memberId, String name) {
```

```
        super (memberId, name);
```

```
    }
```

```
@Override
```

```
    public int getBorrowingLimit () {
```

```
        return 3;
```

```
    }
```

```
class TeacherMember extends member {
```

```
    public TeacherMembers (String memberId, String name) {
```

```
        super (memberId, name);
```

```
    }
```

```
@Override
```

```
    public int getBorrowingLimit () {
```

```
        return 10;
```

```
    }
```

```
}
```

7. Attendance Tracking (Req 7)

```
public class Attendance {
    public double calculateAttendance(int daysPresent, int totalDays) {
        return ((double) daysPresent / totalDays) * 100;
    }
}
```

```
class EngineeringAttendance extends Attendance {
    @Override
    public double calculateAttendance(int daysPresent, int totalDays) {
        return super.calculateAttendance(daysPresent, totalDays);
    }
}
```

```
class MedicalAttendance extends Attendance {
    @Override
    public double calculateAttendance(int daysPresent, int totalDays) {
        return super.calculateAttendance(daysPresent, totalDays);
    }
}
```

8. Notification System (Req 8)

```
public interface Notification {
    void sendNotification(String recipient, String message);
}
```

```
class EmailNotification implements Notification {
    @Override
    public void sendNotification(String recipient, String message) {
        System.out.println("Sending Email to " + recipient + ": " + message);
    }
}
```

```
class SmsNotification implements Notification {
    @Override
    public void sendNotification(String recipient, String message) {
        System.out.println("Sending SMS to " + recipient + ": " + message);
    }
}
```

```
class MmsNotification implements Notification {
    @Override
    public void sendNotification(String recipient, String message) {
        System.out.println("Sending MMS to " + recipient + ": " + message);
    }
}
```

5. Scholarship Allocation (Req. 10)

```
abstract class Scholarship {  
    protected String studentName;
```

```
    public Scholarship(String studentName) {  
        this.studentName = studentName;
```

```
    }  
    public abstract boolean checkEligibility();  
}
```

```
class MeritScholarship extends Scholarship {  
    private double gpa;
```

```
    public MeritScholarship(String studentName, double gpa) {  
        super(studentName);
```

```
        this.gpa = gpa;
```

```
    @Override
```

```
    public boolean checkEligibility() {  
        return gpa >= 3.8;
```

```
    }  
    class NeedBasedScholarship extends Scholarship {  
        private double annualIncome;
```

```
        public NeedBasedScholarship extends Scho
```

```
        public NeedBasedScholarship(String studentName, double annual  
            Income) {
```

```
            super(studentName);
```

```
            this.annualIncome = annualIncome;
```

```
}
```

```
    @Override
```

```
    public boolean checkEligibility() {  
        return annualIncome < 20000;
```

```
    }  
}
```


OUTPUT

```
"C:\Program Files\Java\jdk-26.0.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\idea_rt.jar=55352" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
--- Student Profiles ---
ID: STU1001, Name: Krish, Roll: 101, Faculty: Engineering
ID: STU1002, Name: Sandesh, Roll: 102, Faculty: Management
--- Employee Details ---
Teacher ID: E001, Name: Dr. Smith, Subject: Computer Science
Admin Staff ID: A002, Name: Jane Doe, Dept: Registrar Office
--- Course Registration ---
Courses registered for Charlie:
- Object Oriented Programming
- Data Structures
--- Grading System ---
Eng 85 Marks: Distinction
Mgmt 70 Marks: B Grade
--- Notifications ---
Sending Email to ruben@university.edu: Welcome to the semester!
Sending SMS to 9768400750: Your fee is due.
Process finished with exit code 0
```